

DUO Policy: Learning Fine-Grained Manipulation from Human Videos via Dual-Actor Policy

Mingxi Jia¹ Patton Jiyang Yin¹ Haojie Huang² Ivy Xiao He¹
Xuanru Liu¹ Gefei Liu¹ Dian Wang³ Stefanie Tellex¹

¹Brown University ²Northeastern University ³Stanford University
https://mingxi-jia.github.io/duo_policy_site/

Abstract: Cross-embodiment learning promises a scalable way to leverage human videos to reduce the need for expensive robot data collection. However, the embodiment gap confines the resulting policies to coarse behaviors, and they fail at precise tasks such as insertion. We observe that this gap matters only where precision is required: human data is useful where precision requirements are low, while robot data is disproportionately valuable near precision-critical states. Building on this observation, we propose **DUO Policy** (Dual-Actor Policy): a hand policy pre-trained on human videos handles global, free-space motion, a robot policy trained on sparse in-hand data makes local, precise corrections, and a gate learned from human interventions switches between them. We find that the dual-actor architecture yields substantial advantages over co-training and finetuning: it remains robust as the embodiment error in human data grows, and it generalizes to scene layouts far beyond the coverage of the robot data. On tasks such as coffee pod insertion, desk cleanup, and cluttered candy sorting, DUO Policy outperforms the best baseline by an average of 25.3% in four simulated and 51.7% in three real-world tasks, while achieving $2.75\times$ better data-collection efficiency.

Keywords: Cross-embodiment Learning, Learning from Human Videos, Imitation Learning

1 INTRODUCTION

Cross-embodiment learning promises a scalable solution to use data from other embodiments to break the data bottleneck in imitation learning. Human videos in particular are cheap to collect, can cover the workspace densely, and capture a diverse set of behaviors [1–3]. However, policies learned from these data sources often fail on fine-grained manipulation tasks such as tight-fit insertion, even when co-trained with robot data. The usual explanation is that retargeted human motions differ systematically from what the target robot should do. Does it mean that human videos are useless for learning fine-grained manipulation? We argue that the answer is no.

In this paper, we argue that **the value of cross-embodiment data depends on the state, and is governed by how much precision that state demands**. Let $\varepsilon(s)$ denote the largest action error from which the task can still succeed at state s . Cross-embodiment data introduces a systematic action error η . Where $|\eta| < \varepsilon(s)$, as in reaching, transporting, and approaching, the error is absorbed and

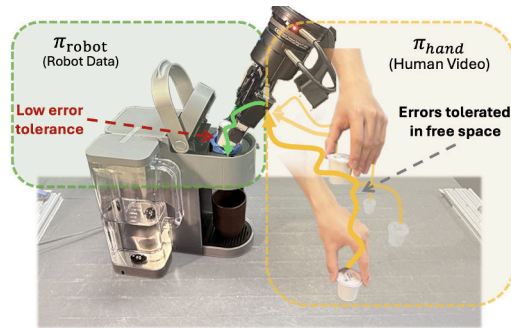


Figure 1: **Where Is Each Data Source Useful?** **Human Hand Data** is imprecise but sufficient in free space; near insertion, only **Robot Data** is precise enough. DUO Policy uses each where it is valuable.

cross-embodiment data contributes what robot data cannot cheaply provide: coverage. Where $\varepsilon(s)$ collapses below $|\eta|$, as in aligning, inserting, and closing a contact, cross-embodiment data becomes actively misleading, and a handful of target-embodiment samples is worth more than any amount of human video. In short, *cross-embodiment data is useful primarily where precision requirements are low. Target-embodiment data is disproportionately valuable near precision-critical states.*

This view explains why existing recipes succeed in general manipulation tasks but struggle with fine-grained manipulation. Phantom [1] fills the visual gap between human and robot by inpainting, but the gap remains in the action space. Co-training methods [2, 3] treat each data source as uniformly useful across states, which confuses the two sources in precision-critical states. Target-supervised finetuning [4, 5] spends scarce robot data relearning free-space behavior the human prior already provides and risks forgetting that prior.

We turn this principle into **DUO Policy** (Dual-Actor Policy). A hand policy pretrained on human video handles the low-precision portion of the task. A robot policy trained on sparse, in-hand robot data handles the precision-critical portion. A learned policy gate switches between them. The gate does not need hand-designed phase labels. We collect robot data with human-in-the-loop interventions, and the moments where an operator takes over mark the states where the human prior’s error exceeds the tolerance. Intervention labels therefore serve as a free proxy for precision-criticality, and the operator’s effort is spent where it matters. Our contributions can be summarized as follows:

- **A state-dependent account of cross-embodiment data value.** We formalize precision-critical states through the tolerance $\varepsilon(s)$ and find that each data source helps where this account predicts. With human data covering the workspace, DUO succeeds on 87.5% of placements versus 31.2% for DP3 trained on the same data, and it degrades the least as embodiment noise grows, whereas co-training on both sources loses 19.9%.
- **DUO Policy, a dual-actor architecture that allocates each data source to the states where it is valuable.** An intervention-supervised gate routes control between a human-pretrained actor and a robot actor. The learned gate matches a privileged phase gate (78/79% vs. 72/73% on seen/unseen placements), while a distance-threshold gate reaches only 49/44%. Local perception and gripper-frame actions are critical: replacing them reduces success from 91.6% to 17.3% and 63.7%.
- **Empirical gains.** DUO Policy outperforms the best baseline by 25.3% on four simulated tasks and 51.7% on three real-world tasks, and reaches 90% success on candy sorting with $2.75\times$ less data-collection time than robot-only training.

2 RELATED WORK

Visuomotor Policy Learning: Because of the high-dimensionality of the raw sensor data, learning visuomotor policies is a challenging problem. Prior works tackle this challenge by end-to-end behavior cloning from robot demonstrations [6, 7]. However, these methods require a large amount of robot data. To improve data efficiency, prior work has explored using 3D representations [8], symmetry [9–16], or data augmentation [17, 18]. However, these methods still require a large amount of robot data to learn complex manipulation tasks. In this paper, we propose leveraging human videos for pretraining, which significantly reduces the amount of robot data required.

Learning from Human Videos: Human videos [19–24] are a valuable alternative data source for learning visuomotor policies [3, 25, 26]. Phantom [1] uses human videos only by overlaying fake arms on RGB frames. Motion Tracks [27] uses keypoints as an intermediate representation to align human and robot actions. Masquerade [2] uses human-robot co-training to improve performance by leveraging human videos and robot data. Nonetheless, these methods train a policy using a mixture of heterogeneous datasets and a unified action space, which undermines the policy’s ability to learn fine-grained actions due to embodiment gaps. In contrast, our method explicitly decouples the low-precision and precision-critical parts of a task, allowing us to fully leverage human data while accounting for the precision effects of noisy cross-embodiment data.

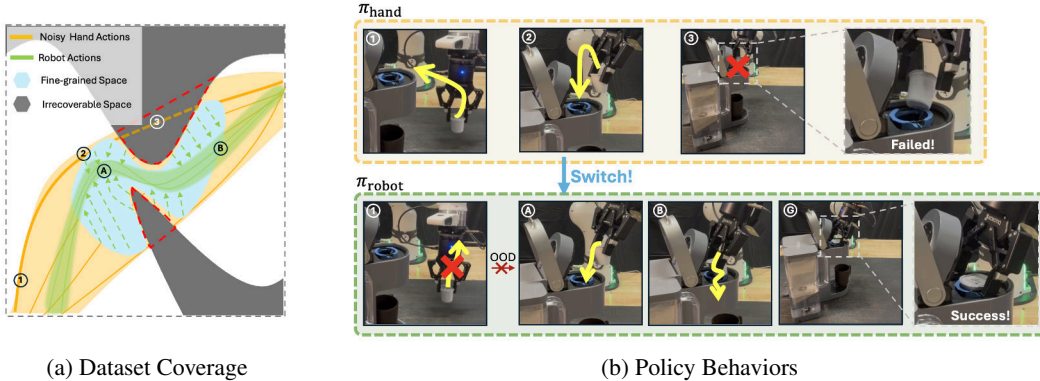


Figure 2: **Each Data Source Is Valuable in Different States.** (a) **Human hand data** covers the workspace, but its embodiment error leads into **irrecoverable** states in the narrow precision-critical region (blue). **Robot data** is precise but sparse. (b) π_{hand} collides during insertion ③, and π_{robot} fails from an out-of-distribution start ①. DUO Policy follows π_{hand} through free space ①–②, switches to π_{robot} near the holder ③–④, and completes the insertion ⑤.

Human-in-the-Loop Policy Learning: Leveraging human corrections is an effective strategy for mitigating covariate shift in imitation learning [28–30]. Recent work has explored incorporating human interventions to improve policy robustness and recovery behaviors. Sirius [31] employs weighted behavioral cloning to prioritize corrective demonstrations, while CR-Dagger [32] augments a base policy with a corrective residual policy. However, these approaches either continue training on imperfect policy-generated data or condition corrections on the base policy’s actions, which can propagate existing errors. In contrast, our method fully decouples the robot policy from the frozen human prior, allowing the policy to learn independently from human corrective guidance without inheriting biases from the original policy.

3 PROBLEM SETTING

We consider learning a fine-grained manipulation policy from two heterogeneous sources: a large, low-cost human-video dataset $\mathcal{D}_h = \{\tau_i^h\}_{i=1}^{N_h}$ and a small, interactive robot dataset $\mathcal{D}_r = \{\tau_j^r\}_{j=1}^{N_r}$. Each timestep provides a global observation o_t^{global} , a scene point cloud fused from calibrated external RGB-D cameras, and, on the robot, a local observation o_t^{local} , an RGB image from a wrist-mounted in-hand camera; human videos provide only o_t^{global} . After retargeting human hand motion into the robot’s end-effector action space, we view $\mathcal{D}_h$ as demonstrations from a biased expert whose actions satisfy $a_t = \pi^*(o_t^{\text{global}}) + \eta(o_t^{\text{global}})$, where η represents the systematic cross-domain shift. The robot dataset $\mathcal{D}_r = \{(o_t^{\text{local}}, a_t^*, i_t)\}$ is collected interactively, where the intervention label $i_t \in \{0, 1\}$ marks the fine-grained timesteps at which a human takes over to supply a precise action a_t^* . Our objective is to learn a policy that maximizes task success while minimizing robot-data collection time. Let $\varepsilon(s)$ be the precision tolerance at state s , i.e., the largest action error from which the task can still succeed. We call $\mathcal{S}_{\text{crit}} = \{s : \varepsilon(s) < |\eta(s)|\}$ the *precision-critical* states, and the remaining states free space.

4 METHOD

DUO Policy builds on the observation in Figure 2: a fine-grained task combines a *global* problem, reaching the right place in a varying scene where centimeter errors are harmless, and a *local* problem, aligning precisely where millimeters matter. Human videos suit the first, since external cameras see the whole scene and free space tolerates their embodiment error η . Sparse robot data with an in-hand camera suits the second. DUO therefore separates the two sources in data, observation, and action: π_{hand} learns from $\mathcal{D}_h$, sees the global scene, and predicts absolute poses, while π_{robot} learns

only from $\mathcal{D}_r$, sees the in-hand view, and predicts gripper-frame corrections. A gate learned from intervention labels decides which actor is in control.

4.1 Two Actors, One Gate

As shown in Figure 3, DUO Policy consists of a hand policy π_{hand} , pretrained on human or any cross-embodiment data, and a robot policy π_{robot} , trained on sparse robot data. A state-dependent binary gate $g(o_t^{\text{local}})$ selects which branch acts at each step:

$$\pi(o_t) = g(o_t^{\text{local}}) \pi_{\text{robot}}(o_t^{\text{local}}) + (1 - g(o_t^{\text{local}})) \pi_{\text{hand}}(o_t^{\text{global}}) \quad (1)$$

When $g = 0$, the robot is in free space and follows the human prior; when $g = 1$, it has entered a precision-critical state and the robot policy takes over. The branches share no parameters, so the human prior’s error η cannot leak into the robot policy, and each branch can use its own observation and action space.

Global / local separation. The hand policy observes the global point cloud o_t^{global} and predicts absolute world-frame poses. Because its observations and actions share the world frame, rotating a demonstration’s point cloud about the gravity axis by $R \in SO(2)$ and applying the same R to its target poses yields another valid demonstration. This $SO(2)$ augmentation trains π_{hand} to be approximately equivariant to the global layout. The gate and the robot policy observe only the in-hand view o_t^{local} , and the robot policy predicts relative motions in the current gripper frame with its own action normalization (Appendix C). This separation gives the policy a useful symmetry.

Proposition 1 (Layout equivariance). *Let $T_g \in SE(2)$ denote a global spatial transformation of the environment layout, respecting the gravity vector, that acts on the state s_t by transforming the scene and the gripper pose together. Let $o_t^{\text{local}} = h(s_t)$ denote the in-hand view, with $h(T_g \cdot s_t) = h(s_t)$. Then the gate is invariant to global scene transformations, $g(h(T_g \cdot s_t)) = g(h(s_t))$, and, with π denoting the induced world-frame end-effector motion and $T_g \cdot$ acting on motions by conjugation, the complete system response is*

$$\pi(T_g \cdot s_t) = \begin{cases} T_g \cdot \pi_{\text{robot}}(h(s_t)), & \text{if } g = 1 \quad (\text{Globally Equivariant}) \\ \pi_{\text{hand}}(T_g \cdot o_t^{\text{global}}), & \text{if } g = 0 \quad (\text{Globally Variant}) \end{cases} \quad (2)$$

Because the gate operates on the local view, its state is inherently invariant to the layout. And because π_{robot} outputs actions in the local gripper frame, this invariant observation mapping naturally induces globally equivariant end-effector motion (proof in Appendix A). This formulation highlights a key sample-efficiency property: when activated ($g = 1$) in precision-critical states, the local perception becomes invariant to global nuisance variables, allowing the induced end-effector motion to remain globally equivariant. The sparse robot data is thus preserved entirely for sample-efficient local geometry learning (e.g., pod insertion) rather than being wasted on learning global spatial variations. Conversely, in free space ($g = 0$), the policy remains variant to global coordinates, leaving the hand policy to absorb workspace variations through dense human data and $SO(2)$ augmentation. Giving the robot branch global observations or absolute actions breaks this property (Table 4).

4.2 Canonical Observations for Cross-Embodiment Transfer

π_{hand} is trained on human observations but deployed on the robot. We remove this mismatch by construction with a canonicalization map ϕ , which removes whichever embodiment acts in the scene and renders the target gripper at its end-effector pose. Human and robot observations thus share one

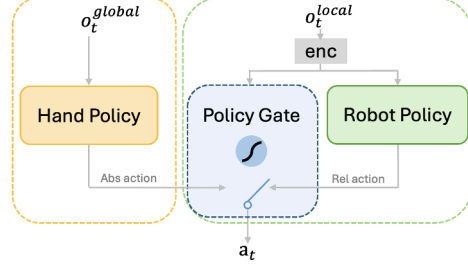


Figure 3: **Architecture.** The hand policy maps the global observation to absolute actions; the robot policy maps the in-hand view to relative actions. The policy gate shares the robot policy’s encoder and switches which actor controls the robot.

Method	Obs	Stack D1			Stack Three D1			HammerCleanup D1			Coffee D2		
		20	30	40	20	30	40	20	30	40	20	30	40
DUO	PCD	88 (-7)	95 (=)	95 (=)	35 (+15)	50 (+20)	50 (+15)	55 (+45)	60 (+47)	65 (+52)	90 (+42)	93 (+45)	90 (+30)
DP3 [8]	PCD	95	95	95	20	20	23	10	8	13	48	45	60
DiffPo [7]	RGB	83	88	88	10	30	35	8	13	10	18	30	33
Phantom [1]	RGB	—	50	—	—	3	—	—	0	—	—	0	—
Phantom-SFT [1]	RGB	60	73	78	5	10	13	5	5	3	5	10	15
Masquerade [2]	RGB	13	38	63	3	8	5	3	5	3	18	48	53

Table 1: **Simulation Results.** Success rates (%) after pretraining on 300 noisy **Panda demonstrations** (500 for HammerCleanup) and finetuning on 20, 30, or 40 **UR5 demonstrations**. Parentheses show the difference from the best baseline; bold indicates the best.

space: π_{hand} is trained on $\mathcal{D}_h = \{(\phi(o_t^h), \hat{a}_t)\}$, where $\hat{a}_t$ is the human hand pose retargeted to a gripper pose, and deployed on $\phi(o_t^r)$. Because ϕ only needs the end-effector pose of the acting embodiment, it applies to any source embodiment whose end-effector pose can be estimated. We parameterize π_{hand} as a conditional diffusion policy [7] over the global point cloud, trained with the standard denoising objective. For human videos, ϕ estimates the hand pose with WiLoR [33] from three calibrated RGB-D cameras and segments out the human. On the robot, it removes the arm using URDF-based filtering and runs at roughly 33 Hz (Appendices B and F). Without this alignment, success drops by 28.3% (Section 5.4).

4.3 Interventions as Labels for Precision-Critical States

The gate must recognize $\mathcal{S}_{\text{crit}}$, but the tolerance $\varepsilon(s)$ that defines it is unknown. We obtain labels from π_{hand} itself. We deploy π_{hand} on the robot in a single round, and an operator takes over whenever it is about to fail, i.e., when its error exceeds what the state tolerates. Labeling each timestep with $i_t = 1$ under operator control and $i_t = 0$ otherwise gives $i_t \approx \mathbb{1}[s_t \in \mathcal{S}_{\text{crit}}]$ at no extra labeling cost, and concentrates the operator’s effort where precision is required. We train π_{robot} , a separate diffusion policy, on all timesteps of $\mathcal{D}_r$, and train the gate g to predict i_t ; both share an in-hand encoder initialized with pre-trained ResNet-18 [34] weights:

$$\mathcal{L}_{\text{finetune}} = \mathcal{L}_{\text{diff}}(\pi_{\text{robot}}; \mathcal{D}_r) + \lambda \mathcal{L}_{\text{CE}}(g(o_t^{\text{local}}), i_t), \quad (3)$$

where λ balances the two losses. The learned gate agrees with a privileged phase label (F1 0.86) and matches its task success (Section 5.3).

5 EXPERIMENTS

We evaluate DUO Policy in simulation and the real world to answer four questions: **(Q1)** Does DUO outperform prior cross-embodiment methods? **(Q2)** Where does each data source help? **(Q3)** Does the gate find precision-critical states? **(Q4)** Which components matter?

Setup. In simulation, we use four MimicGen [35] tasks. We compare to cross-embodiment baselines. Phantom [1] is a zero-shot method which is trained on inpainted human demonstrations without robot data. We further implement a Phantom-SFT variant, which allows for fine-tuning on more robot data. Masquerade [2] co-trains robot actions with auxiliary human keypoints. For behavioral cloning baselines, we compare to Diffusion Policy [7] and 3D Diffusion Policy (DP3) [8]. To ensure fair comparison, we pretrain the baselines with noisy pretraining data, then finetune with perfect robot data. Two key constraints apply to our pretraining datasets: (1) as in the real-world human-video scenario, they contain no in-hand camera views, and (2) we simulate the systematic embodiment gaps by adding random noise onto pretrain trajectories. In the real world, we evaluate three tasks (Figure 4) and compare against DP3, which we allow to use additional in-hand images as well as the point clouds, and Masquerade-3D, a Masquerade-style baseline that co-trains on 3D human and robot demonstrations. Each analysis uses its own evaluation set and checkpoint selection (e.g., 80 placements for Figure 6 and separate seen and unseen placements for Table 3), so success rates are comparable within a table or figure but not across them.

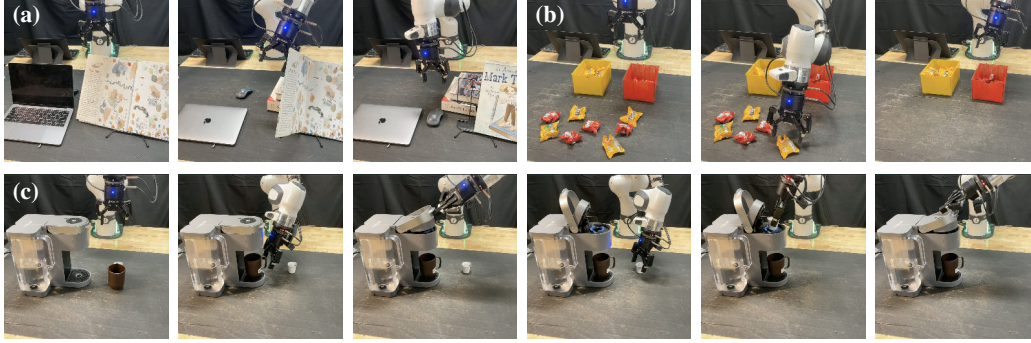


Figure 4: **Real-world Tasks.** (a) **Desk Cleanup:** close the laptop and the book, then place the mouse aligned with the laptop. (b) **Candy Sorting:** sort cluttered candy bags into boxes by color. (c) **K-Cup Coffee Making:** place the mug, open the machine, insert the pod, close the machine, and press brew.

5.1 Does DUO Outperform Prior Methods?

Simulation. As shown in Table 1, our method significantly outperforms the baselines in all environments, except Stack D1. Stack D1 has a loose precision requirement, so a policy pretrained on the noisy cross-embodiment data already achieves a high success rate and leaves little margin for the robot branch. This is consistent with our hypothesis: cross-embodiment data is sufficient where precision requirements are low, and the gains of DUO grow on tasks with precision-critical states such as Coffee D2 and HammerCleanup D1.

Real World. We design three real-world tasks that require fine-grained manipulation skills as shown in Figure 4. The first task is Desk Cleanup, where the robot needs to carefully close the laptop without colliding and precisely align the mouse with the laptop. The second task is Candy Sorting, which contains eight candy bags and two boxes which are randomly initialized on the table. It is challenging because (1) the candy can appear in widely varying poses and arrangements, and (2) the robot must reason about a “same color goes together” rule to sort correctly. The K-Cup coffee-making task is the most complicated one because it is very fine-grained and long-horizon. As shown in Table 2, our method outperforms both baselines in all three tasks. Especially for the coffee-making task, our method achieves a 70% success rate while both baselines achieve only 10%. Compared to DP3, co-training in Masquerade-3D improves Candy Sort (47/80 vs. 28/80) but degrades Desk Cleanup (0/20 vs. 9/20). Candy Sort benefits from the broader configuration coverage of human data, whereas the fine-grained tasks fail because the policy is co-trained on noisy human actions. Naive co-training thus improves coverage but compromises precision, consistent with our thesis; DUO obtains the coverage without the loss of precision.

Method	Desk Cleanup	Candy Sort	K-Cup Coffee
DUO	17/20	72/80	14/20
DP3	9/20	28/80	2/20
Masquerade-3D	0/20	47/80	2/20

Table 2: **Real-world Task Success Rates.**

Data Time Efficiency. Here, we investigate the data collection efficiency of our method. One key advantage of learning from human demonstrations is the ability to collect data quickly and efficiently, without compromising fine-grained control. We chose the candy-sorting task for this investigation because its wide range of candy-bag configurations demands a substantial amount of data to learn. For our method, we first spend 10 minutes collecting human hand demonstrations, yielding 216 human hand trajectories. We then train the hand policy and use it for human-in-the-loop data collection. After 30 additional minutes of HG-Dagger, our method achieves a 90% success rate. Under the same 40-minute data collection budget, DP3 with robot demonstrations collected via SpaceMouse only reaches a 35% success rate. To achieve a competitive

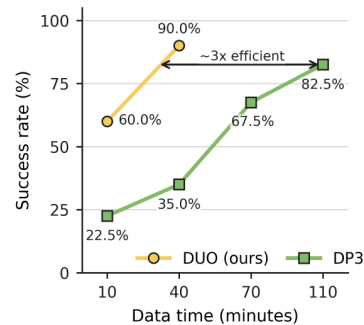


Figure 5: **Data Collection Time.** X/Y-axis shows the wall-clock data collection time and success rates.

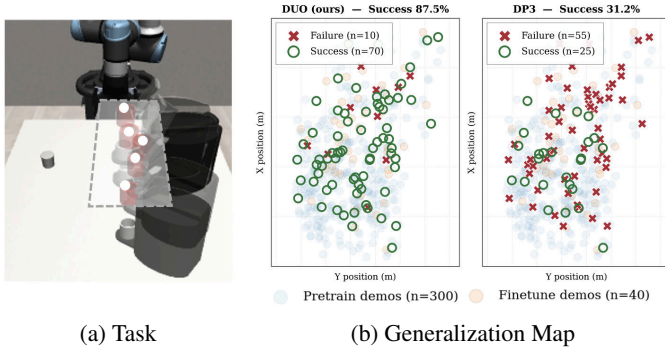


Figure 6: **Spatial Generalization.** (a) Only the coffee machine pose varies within the gray region. (b) Insertion outcomes over placements, with pretraining and finetuning demonstrations in the background.

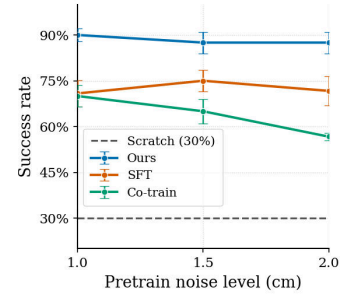


Figure 7: **Robustness to Embodiment Noise.** Success on Coffee D2 as pretraining noise grows.

success rate, DP3 requires another 70 minutes of robot data collection. This result validates our data-collection efficiency, which leverages low-cost human demonstrations to achieve high performance.

5.2 Where Does Each Data Source Help?

Our hypothesis predicts that human data contributes coverage where precision requirements are low, while robot data must govern precision-critical states. We test both halves on Coffee D2.

Human data provides coverage. Figure 6 tests the first half of our thesis on a coffee-pod insertion task in which only the pose of the coffee machine varies. The 300 noisy pretraining demonstrations cover the workspace densely, while the 40 robot demonstrations are sparse. DUO succeeds on 87.5% of 80 test placements, compared to 31.2% for DP3 trained on the same data. The failure patterns differ: DP3 succeeds mainly near the robot demonstrations, whereas DUO succeeds almost everywhere the pretraining data reach, and its few failures are scattered rather than clustered. This is the behavior our hypothesis predicts. The embodiment error in the pretraining data is tolerated in free space, so the hand policy brings the gripper to the holder from anywhere in the workspace. Once there, the in-hand view looks the same regardless of where the machine sits, so 40 robot demonstrations suffice to learn the local insertion. DP3 instead relies on a single finetuned policy, whose precise behavior holds only near the robot data.

Separate actors isolate the embodiment error. We inject uniformly sampled noise of increasing magnitude into each pretraining trajectory on Coffee D2 to simulate a growing embodiment gap (Figure 7). DUO degrades the least because its robot branch never trains on the noisy data. Co-training, which forces a single policy to fit both sources, deteriorates as noise grows; SFT stays robust but never becomes competitive; training from scratch cannot use the pretraining data at all. Training curves are in Appendix D.

5.3 Does the Gate Find Precision-Critical States?

To test whether the gate discovers precision-critical states, we keep both trained branches fixed and replace only the gate (Table 3). Always-hand and always-robot use a single branch throughout. The privileged phase gate switches on the task-phase label from MimicGen’s subtask segmentation [35], which uses privileged simulator state, and the distance gate switches when the end-effector is within r cm of the target, using the privileged object pose. We evaluate on the spatial-generalization layout (Figure 6), separating placements inside (seen)

Gate	Seen	Unseen	F1
Always hand	0	0	–
Always robot	41	33	0.70
Distance [†]	49	44	0.77
Privileged phase [†]	72	73	1.00
Learned (ours)	78	79	0.86

Table 3: **Gate Analysis on Coffee D2.** Success rate (%), mean over 3 seeds) on spatial-layout placements inside (Seen) and outside (Unseen) the robot-data region. F1 is against the privileged phase label. [†]Privileged state.

and outside (unseen) the region covered by the robot data. The learned gate matches or slightly exceeds the privileged phase gate (78 vs. 72% seen, 79 vs. 73% unseen) and agrees with it on most timesteps (F1 0.86). A phase label marks where the task changes, not where π_{hand} actually fails; the learned gate is trained on interventions, which mark the latter, so it can hand over control earlier or later as the hand policy’s errors require. Its success does not drop on unseen placements, because the in-hand gate only needs to recognize local geometry. In contrast, the distance gate switches to the robot branch too early, so it behaves much like always-robot and inherits its weakness: the robot branch must also cover approach motion, and success falls from 49% to 44% on unseen placements (41% to 33% for always-robot). Always-hand never completes the task, confirming that the human prior alone cannot handle the precision-critical states.

5.4 Ablations

As shown in Table 4, we ablate each component. **Dual actors.** Replacing the dual actors with a single policy trained by SFT or co-training drops performance by 4.9% and 19.9%, consistent with Section 5.2: co-training lets the embodiment error leak into precision-critical states, and SFT gradually forgets the human prior. **Human prior.** Without human pretraining, success drops by 63.3%, since 40 robot demonstrations alone cannot cover the workspace. Without the canonicalization ϕ , which renders the target (Robotiq) gripper into the pretraining observations, it drops by 28.3%

because of the visual gap between pretraining and deployment observations. Replacing the 3D point cloud with image input drops it by 49.1%: without in-hand views, the hand policy needs 3D geometry to learn actionable behaviors from human videos. **Global-local separation.** Giving π_{robot} the global observation instead of the in-hand view reduces success to 17.3%, and predicting absolute instead of gripper-frame actions reduces it to 63.7%. Both changes break Proposition 1, so the sparse robot data must also cover layout variation; the local observation matters most.

Component	Variant	Success	Δ
Dual actors (Sec. 4.1)	Single policy, SFT	86.7	−4.9
	Single policy, co-training	71.7	−19.9
Human prior (Sec. 4.2)	No human pretraining	28.3	−63.3
	No canonicalization ϕ	63.3	−28.3
	Point cloud $\rightarrow$ image	42.5	−49.1
Global / local (Prop. 1)	Local $\rightarrow$ Global obs. for π_{robot}	17.3	−74.3
	Rel $\rightarrow$ Abs actions for π_{robot}	63.7	−27.9
Full DUO		91.6	–

Table 4: **Ablation Study on Coffee D2.** Success rates (%) averaged over three seeds; Δ is relative to Full DUO.

6 Conclusion and Limitations

In this paper, we argue that cross-embodiment data is useful primarily where precision requirements are low, while target-embodiment data is disproportionately valuable near precision-critical states. We present DUO Policy, a dual-actor policy architecture that turns this principle into practice for learning fine-grained manipulation policies. Our method achieves high data efficiency and precision by using a binary policy gate, learned from human interventions, to switch between a hand policy pretrained on human data for global, free-space motion and a robot policy trained on sparse in-hand robot data for local, precise corrections. We validate our approach in both simulation and real-world settings, demonstrating significant improvements over several baselines, strong spatial generalization beyond the robot data, and a learned gate that matches a privileged phase gate. We outperform the best baseline by an average of 25.3% in four simulation tasks and 51.7% in three challenging real-world tasks, while achieving 2.75 $\times$ better data-collection efficiency. One major limitation of our method is that it works best with a full point cloud via multi-view RGB-D cameras. The performance drops by 14.9% when using a single-view point cloud. The main failure mode is that, without multi-view information, the policy struggles with occlusions and fails to generalize in some unseen scenarios. In the future, we will explore improving the performance with single-view point clouds and further scale our method with open-source egocentric datasets, which often consist of single-view observations.

References

- [1] M. Lepert, J. Fang, and J. Bohg. Phantom: Training robots without robots using only human videos. In *Conference on Robot Learning*, pages 4545–4565. PMLR, 2025.
- [2] M. Lepert, J. Fang, and J. Bohg. Masquerade: Learning from in-the-wild human videos using data-editing. *arXiv preprint arXiv:2508.09976*, 2025.
- [3] S. Kareer, D. Patel, R. Punamiya, P. Mathur, S. Cheng, C. Wang, J. Hoffman, and D. Xu. EgoMimic: Scaling imitation learning via egocentric video. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13226–13233. IEEE, 2025.
- [4] R. Yang, Q. Yu, Y. Wu, R. Yan, B. Li, A.-C. Cheng, X. Zou, Y. Fang, X. Cheng, R.-Z. Qiu, H. Yin, S. Liu, S. Han, Y. Lu, and X. Wang. EgoVLA: Learning vision-language-action models from egocentric human videos. *arXiv preprint arXiv:2507.12440*, 2025.
- [5] L. Wang, X. Chen, J. Zhao, and K. He. Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2024.
- [6] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.
- [7] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [8] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. *Robotics: Science and Systems XX*, 2024.
- [9] D. Wang, R. Walters, and R. Platt. SO(2)-equivariant reinforcement learning. In *International Conference on Learning Representations*, 2022.
- [10] H. Huang, D. Wang, A. Tangri, R. Walters, and R. Platt. Leveraging symmetries in pick and place. *The International Journal of Robotics Research*, 43(4):550–571, 2024.
- [11] D. Wang, S. Hart, D. Surovik, T. Kelestemur, H. Huang, H. Zhao, M. Yeatman, J. Wang, R. Walters, and R. Platt. Equivariant diffusion policy. In *8th Annual Conference on Robot Learning*, 2024.
- [12] B. Hu, D. Wang, D. Klee, H. Tian, X. Zhu, H. Huang, R. Platt, and R. Walters. 3D equivariant visuomotor policy learning via spherical projection. *Advances in Neural Information Processing Systems*, 38:5685–5715, 2026.
- [13] M. Jia, H. Huang, Z. Zhang, C. Wang, L. Zhao, D. Wang, J. X. Liu, R. Walters, R. Platt, and S. Tellex. Learning efficient and robust language-conditioned manipulation using textual-visual relevancy and equivariant language mapping. *IEEE Robotics and Automation Letters*, 2025.
- [14] D. Wang, S. Hart, D. Surovik, T. Kelestemur, H. Huang, H. Zhao, M. Yeatman, X. Zhu, B. Hu, M. Jia, et al. Equivariant diffusion policy for sample-efficient robotic manipulation. *The International Journal of Robotics Research*, page 02783649261424445, 2026.
- [15] H. Huang, L. Zhao, H. Liu, Z. Ye, S.-Y. Huang, M. Jia, B. Hu, F. Lin, Y. Qi, D. Wang, et al. Pix2Act: Image-space manipulation policies with equivariant augmentation. *arXiv preprint arXiv:2607.11167*, 2026.
- [16] H. Huang, Z. Ye, L. Zhao, B. Hu, M. Jia, Y. Qi, A. Agha, D. Wang, R. Platt, and R. Walters. Action map policy: Learning 3D closed-loop manipulation via pixel classification. *arXiv preprint arXiv:2607.10706*, 2026.

- [17] M. Jia, D. Wang, G. Su, D. Klee, X. Zhu, R. Walters, and R. Platt. SEIL: Simulation-augmented equivariant imitation learning. In *IEEE International Conference on Robotics and Automation*. IEEE, 2023.
- [18] G. Papagiannis and E. Johns. MILES: Making imitation learning easy with self-supervision. In *Conference on Robot Learning*, pages 810–829. PMLR, 2025.
- [19] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18995–19012, 2022.
- [20] R. Fu, D. Zhang, A. Jiang, W. Fu, A. Funk, D. Ritchie, and S. Sridhar. GigaHands: A massive annotated dataset of bimanual hand activities. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17461–17474, 2025.
- [21] K. Chen, C. Min, L. Zhang, S. Hampali, C. Keskin, and S. Sridhar. FoundHand: Large-scale domain-specific learning for controllable hand image generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17448–17460, 2025.
- [22] C. Min and S. Sridhar. GenHeld: Generating and editing handheld objects. *arXiv preprint arXiv:2406.05059*, 2024.
- [23] H. Li, W. Fu, X. Cong, Z. Li, B. Huang, H. Jiang, X. He, Y. Liang, R. Fu, T. Lu, et al. Deform360: A massive multi-view visuotactile dataset for deformable world models. In *European Conference on Computer Vision*, pages 152–172. Springer, 2026.
- [24] C. Min, P. Muna-McQuay, T. Lu, J. Tompkin, and S. Sridhar. MotionSplicer: Part-based motion editing for 4D volumetric videos. In *European Conference on Computer Vision*, pages 552–572. Springer, 2026.
- [25] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar. MimicPlay: Long-horizon imitation learning by watching human play. In *Conference on Robot Learning*, pages 201–221. PMLR, 2023.
- [26] R. Zheng, D. Niu, Y. Xie, J. Wang, M. Xu, Y. Jiang, F. Castañeda, F. Hu, Y. L. Tan, L. Fu, et al. EgoScale: Scaling dexterous manipulation with diverse egocentric human data. *arXiv preprint arXiv:2602.16710*, 2026.
- [27] J. Ren, P. Sundaresan, D. Sadigh, S. Choudhury, and J. Bohg. Motion tracks: A unified representation for human-robot transfer in few-shot imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8802–8810. IEEE, 2025.
- [28] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [29] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer. HG-Dagger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083. IEEE, 2019.
- [30] Y. Korkmaz and E. Bıyık. MILE: Model-based intervention learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15673–15679. IEEE, 2025.

- [31] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. *The International Journal of Robotics Research*, 44(10-11):1727–1742, 2025.
- [32] X. Xu, Y. Hou, Z. Liu, and S. Song. Compliant residual DAgger: Improving real-world contact-rich manipulation with human corrections. *Advances in Neural Information Processing Systems*, 38:139559–139581, 2026.
- [33] R. A. Potamias, J. Zhang, J. Deng, and S. Zafeiriou. WiLoR: End-to-end 3D hand localization and reconstruction in-the-wild. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12242–12254, 2025.
- [34] K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.
- [35] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning*, pages 1820–1864. PMLR, 2023.
- [36] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni, R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.

A Proof of Proposition 1

Setup. Let $T_t \in SE(3)$ be the world-frame gripper pose. The robot policy outputs a relative motion $\Delta_t = \pi_{\text{robot}}(o_t^{\text{local}}) \in SE(3)$ in the current gripper frame, executed as $T_{t+1} = T_t \Delta_t$. Let $T_g \in SE(2) \subset SE(3)$ be a gravity-preserving rigid transformation of the scene layout. We assume the in-hand view is a function of the scene relative to the gripper, i.e., $o_t^{\text{local}} = h(s_t) = \tilde{h}(T_t^{-1} \cdot \mathcal{S})$ for scene geometry $\mathcal{S}$, where the state s_t comprises $\mathcal{S}$ and T_t .

Claim. If the original rollout starts at T_0 in scene $\mathcal{S}$ and $g = 1$ along it, then the rollout in the transformed scene $T_g \cdot \mathcal{S}$ starting at $T_g T_0$ visits $T_g T_0, T_g T_1, \dots$, with the same gate values.

Proof. By induction on t . The base case holds by construction. Suppose the gripper is at $T_g T_t$ in $T_g \cdot \mathcal{S}$. Its view is $\tilde{h}((T_g T_t)^{-1} T_g \cdot \mathcal{S}) = \tilde{h}(T_t^{-1} \cdot \mathcal{S})$, identical to the original view; that is, $h(T_g \cdot s_t) = h(s_t)$. Hence the gate value and the relative motion Δ_t are unchanged, and the next pose is $T_g T_t \Delta_t = T_g T_{t+1}$. $\square$

Connection to Eq. (2). The world-frame end-effector motion induced at step t is $M_t = T_t \Delta_t T_t^{-1}$, so that $T_{t+1} = M_t T_t$. In the transformed scene it becomes $(T_g T_t) \Delta_t (T_g T_t)^{-1} = T_g M_t T_g^{-1}$, i.e., the original motion transformed by conjugation with T_g . This is the action $\mathbf{T}_g \cdot \pi_{\text{robot}}$ in Eq. (2).

Remarks. (1) The assumption holds approximately in practice, since the in-hand view may include background that does not move with T_g . (2) With absolute actions, the policy outputs a world-frame target T_t^* that must itself become $T_g T_t^*$ under the transformation; the network would have to learn this mapping from sparse robot data. (3) With global observations, the input changes under T_g , so the invariance of g and π_{robot} is no longer guaranteed.

B Human Hand Data Preprocessing

Action Preprocessing: Hand pose tracking can be jerky and unreliable due to sensor noise, occlusion, etc. It is crucial to ensure high-quality 6-DoF hand poses for policy pretraining. Here, we introduce a pipeline (see Fig. B.1) to obtain them. First, we obtain raw hand poses using WiLoR [33] and filter out low-confidence detections. Then, following [1], we use the depth image to correct the translation component of the hand poses in camera coordinates. To reduce jerky motions, we smooth the hand trajectories with a Savitzky–Golay filter. Finally, because the human hand differs in morphology from the robot gripper, we align the recovered hand pose ${}^W T_H$ with the initial gripper pose ${}^W T_G^{\text{init}}$ such that their fingertip positions are aligned. We transform hand poses into the world frame using the camera extrinsics, ${}^W T_H = {}^W T_C {}^C T_H$, and compute a fixed hand-to-gripper offset from the first frame, ${}^H T_G = ({}^W T_{H,0})^{-1} {}^W T_G^{\text{init}}$. Gripper poses for all timesteps are then ${}^W T_{G,t} = {}^W T_{H,t} {}^H T_G$.

Observation Preprocessing: For every camera view, we segment the human data collector with SAM3 [36] using the language prompt “human” and back-project the remaining pixels into a human-free point cloud scene. We then render the target robot gripper into the scene at the retargeted end-effector pose to form the observation o_t at timestep t ; this implements the canonicalization map ϕ of Section 4.2. Finally, we crop the point cloud to the workspace boundaries and downsample it to 1024 points. Combined with the retargeted actions, this yields the processed human dataset $\mathcal{D}_h = \{\bar{\zeta}_1, \dots, \bar{\zeta}_n\}$, where each trajectory $\bar{\zeta}_i = \{(\bar{o}_t^0, \bar{o}_t^1, \bar{o}_t^2, \bar{a}_t)\}_{t=0}^T$ pairs the three-view observation with the corresponding action at every timestep.

Figure B.2 shows examples of the collected hand data and the processed hand trajectories produced by our preprocessing pipeline.

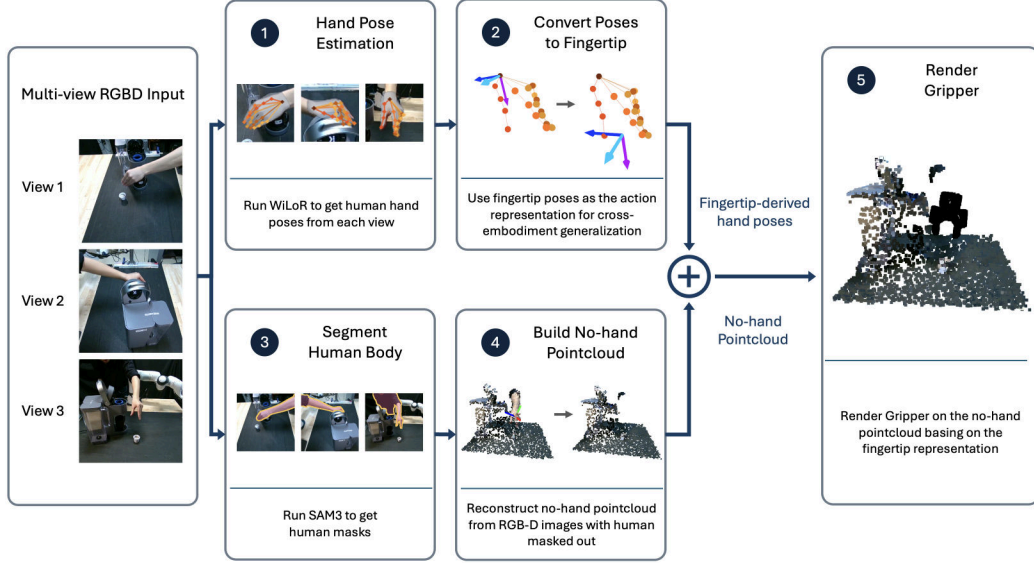


Figure B.1: Data Preprocessing Pipeline.

C Human and Robot Action Scale Comparison

Beyond the visual discrepancy between hand and robot data from the policy observation perspective, we also observed a misalignment in the action distributions. Fig. C.1 shows that robot actions are sharply peaked around zero across all action dimensions, whereas hand actions are more broadly distributed. This suggests that robot trajectories contain many small adjustments, whereas hand trajectories often involve larger per-timestep displacements. One likely reason is that the human collector is more confident and adept when manipulating with their own hands. This causes a temporal gap between the two embodiments, where human actions are often larger than robot actions per timestep. Directly mixing these data can make it difficult for the robot policy to learn the fine-grained micro-actions required for precise manipulation. This motivates the DUO Policy architecture, which separates the human prior from the robot policy.

Decoupled action normalization. Because the two branches share no parameters, each is trained with its own action normalization statistics. Applying a joint normalization $\nu(a) = (a - \mu_{\text{joint}}) / \sigma_{\text{joint}}$ would let the dominant human variance ($\sigma_{\text{hand}} \gg \sigma_{\text{robot}}$) squash the robot actions into a narrow band near zero, dampening the learning signal for fine-grained corrections. Modality-specific statistics $(\mu_{\text{hand}}, \sigma_{\text{hand}})$ and $(\mu_{\text{robot}}, \sigma_{\text{robot}})$ preserve the numerical scale of the sparse robot corrections.

D Training Convergence

Figure D.1 compares the training curves of DUO Policy, SFT, co-training, and training from scratch on Coffee D2. Our method improves faster than the baselines and converges to a higher success rate.

E IK-Aware Human Demonstration Collection

We use WiLoR for hand detection because we found its predictions to be both reliable and computationally efficient for real-time use. During data collection, we observed that, due to inverse kinematics (IK) limits, certain hand motions that are feasible for humans cannot be executed by the robot. To address this issue, we developed a real-time interface that uses WiLoR to estimate hand

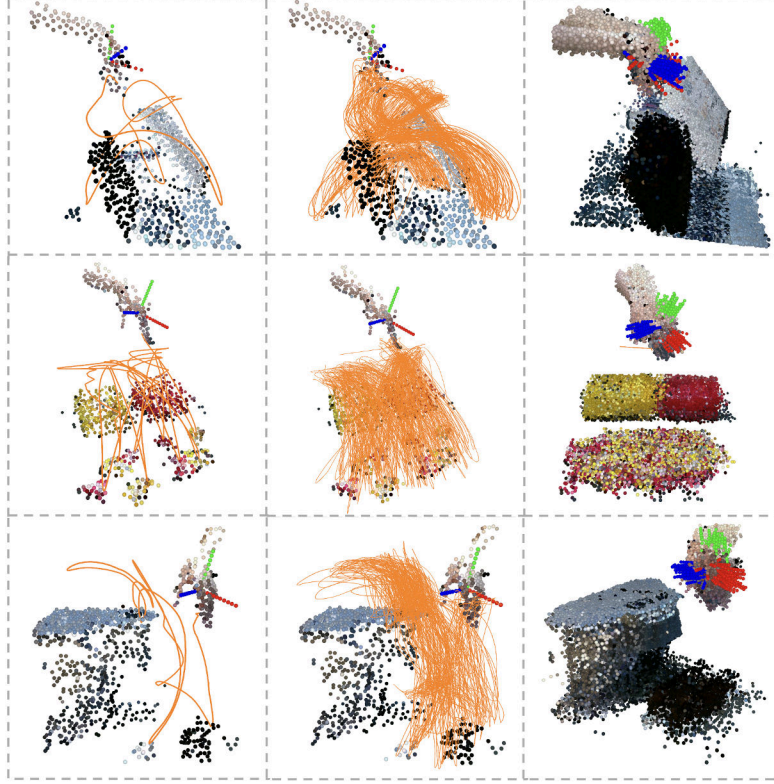


Figure B.2: **Processed Human-Hand Trajectories and Point-Cloud Observations.** We visualize examples of the collected and processed human-hand data across the three real-world tasks. **Column 1:** an example collected human-hand trajectory; **Column 2:** processed hand trajectories aggregated across demonstrations; **Column 3:** domain-randomized point-cloud observations. **Row 1:** Desk Cleanup; **Row 2:** Candy Sorting; **Row 3:** K-Cup Coffee Making.

poses online, transforms the estimated hand pose into a target gripper pose, and computes the corresponding robot joint configuration through inverse kinematics. Whenever the estimated hand motion would result in an infeasible robot configuration, the interface immediately notifies the human operator. This allows the data collector to adjust the motion during the demonstration and reduces the risk of collecting infeasible trajectories.

F Inference-Time Observation Alignment Details

During hand-policy inference, we transform the real robot point-cloud observation into the same representation used for hand-policy training. We first reconstruct a world-frame point cloud from calibrated RGB-D cameras. Given the robot joint positions and calibrated robot base frame, we compute the current robot geometry using forward kinematics on the robot URDF. We then sample the posed robot visual meshes and remove observed point-cloud points whose nearest sampled robot-body point is within a fixed distance threshold (4 cm), using a KD-tree nearest-neighbor query. We exclude gripper finger links from this removal step to avoid erasing manipulated objects near the contact region. Finally, we append a synthetic gripper point cloud generated from a gripper-only URDF at the current gripper state and transformed to the measured end-effector pose. The resulting rendered point cloud contains the scene with the robot arm removed and a standardized rendered gripper, matching the hand-policy training observation distribution.

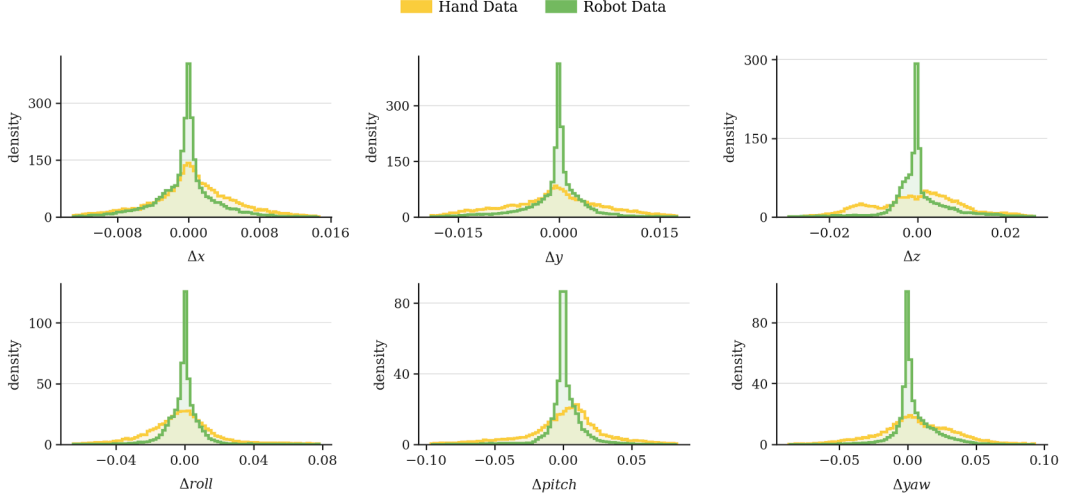


Figure C.1: **Action Distribution Comparison.** Human hand actions vs. robot actions collected with a SpaceMouse (without interventions).

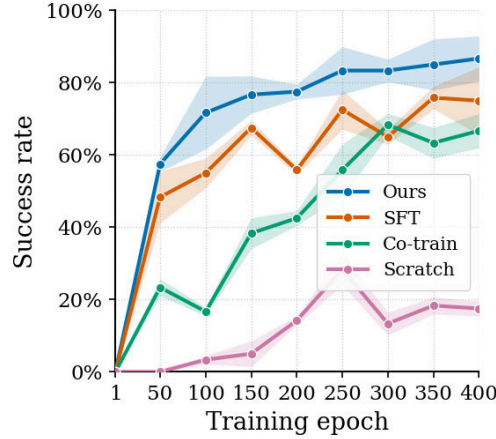


Figure D.1: **Convergence of Training Strategies on Coffee D2.**

G Real-World Task Details

Desk Cleanup: In the Desk Cleanup task, the robot must complete the following sequence of actions: 1. fully close the laptop; 2. fully close the book; 3. pick up the mouse; 4. place it next to the laptop with the correct alignment. An episode is considered successful only if all four steps are completed. Failure at any step results in an unsuccessful episode.

Candy Sorting: In the Candy Sorting task, the robot must sort all eight candy packs into boxes according to their colors. We report the success rate based on the number of candy packs successfully sorted. A sorting attempt is counted as unsuccessful if: 1. the candy pack is placed in the wrong box, e.g., a yellow pack is placed in the red box; 2. the candy pack is not placed inside the correct box, e.g., it falls between two boxes; 3. the robot fails more than three times to pick up the candy pack. If the robot fails more than three times on a given candy pack, the episode is terminated, and all remaining candy packs are counted as unsuccessful.

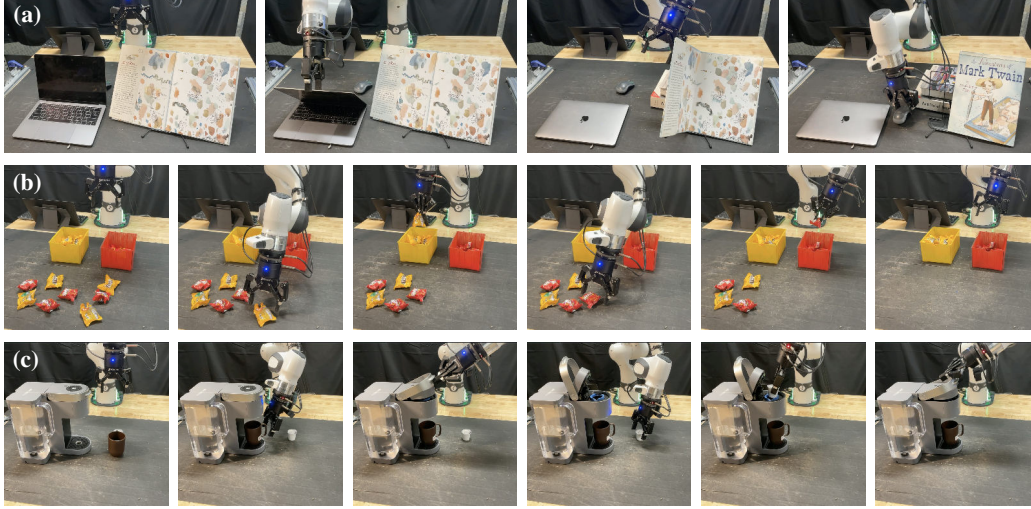


Figure G.1: **Real-world Tasks.** (a) **Desk Cleanup.** The robot needs to close the laptop and the book and then place the mouse closely aligned with the laptop. (b) **Candy Sorting.** The robot needs to sort the cluttered candies by color. (c) **Coffee Making.** The robot needs to complete a series of subtasks to make coffee.

Table H.1: Training hyperparameters used for pretraining and finetuning.

Hyperparameter	Pretrain	Finetune
Observation horizon	1	1
Prediction horizon	16	16
Action execution horizon	8	8
Batch size	256	256
Number of epochs	401	801
Learning rate	1×10^{-4}	1×10^{-4}
Weight decay	1×10^{-6}	1×10^{-6}
LR scheduler	Cosine	Cosine
Warmup steps	500	500

K-Cup Coffee Making: K-Cup Coffee Making is a long-horizon task in which the robot must complete the following sequence of subtasks: 1. pick up the mug; 2. place the mug under the coffee machine; 3. open the coffee machine lid; 4. grasp a coffee pod; 5. insert the coffee pod into the coffee machine; 6. close the coffee machine lid; 7. press the button on the coffee machine. An episode is considered successful only if all subtasks are completed. Failure at any subtask results in an unsuccessful episode.

H Hyperparameters

See Table H.1.